MACHINE LEARNING LAB

Complete Practical Programs — Experiments 1 to 10

Course Code	U24AIMLPC508
Student	Akash Kumar Saw
Roll Number	03
University	D. Y. Patil University
Format	Complete Python programs with dataset descriptions, output notes and results

Datasets used: Iris, Diabetes, Breast Cancer Wisconsin and Wine from scikit-learn. Logic-gate truth tables are used for the neural-network logic-gate experiment because they are mathematical examples.

Experiment 1: Mean, Median, Standard Deviation and Mode

Aim

Write a program to calculate mean, median, standard deviation and mode using user-defined functions.

Dataset and notes

Iris dataset (scikit-learn). The sepal-length measurements are rounded to one decimal place for frequency-based mode calculation. Population standard deviation (division by N) is used.

Complete Python Program

```
from sklearn.datasets import load_iris
from collections import Counter

# Load the real Iris dataset
iris = load_iris()
data = iris.data[:, 0].round(1).tolist()

def calculate_mean(data):
    return sum(data) / len(data)

def calculate_median(data):
    sorted_data = sorted(data)
    n = len(sorted_data)

    if n % 2 == 1:
        return sorted_data[n // 2]
    return (sorted_data[n // 2 - 1] + sorted_data[n // 2]) / 2

def calculate_std(data):
    mean = calculate_mean(data)
    variance = sum((x - mean) ** 2 for x in data) / len(data)
    return variance ** 0.5

def calculate_mode(data):
    frequency = Counter(data)
    max_frequency = max(frequency.values())
    return sorted([
        value for value, count in frequency.items()
        if count == max_frequency
    ])

print("Dataset: Iris Sepal Length")
print("Number of observations:", len(data))
print("Mean:", round(calculate_mean(data), 4))
print("Median:", calculate_median(data))
print("Standard Deviation:", round(calculate_std(data), 4))
print("Mode:", calculate_mode(data))
```

Output / expected result

```
Dataset: Iris Sepal Length
Number of observations: 150
Mean: 5.8433
Median: 5.8
Standard Deviation: 0.8253
Mode: [5.0]
```

Result

Mean, median, population standard deviation and mode were calculated using user-defined functions on real Iris measurements.

Experiment 2: DataFrame Creation and Statistical Summary

Aim

Create a DataFrame with at least five observations, retrieve a column, summarize the data, and calculate mean and standard deviation.

Dataset and notes

First 10 observations from the real Iris dataset. The full output is printed when the program runs.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_iris

# Load the real Iris dataset
iris = load_iris(as_frame=True)

# Select the first 10 observations
df = iris.frame.head(10).copy()
df["Species"] = df["target"].map(dict(enumerate(iris.target_names)))
df.drop(columns=["target"], inplace=True)

print("DataFrame:")
print(df.to_string(index=False))

print("\nSepal Length Column:")
print(df["sepal length (cm)"].to_string(index=False))

print("\nSummary Statistics:")
print(df.describe())

print("\nMean:")
print(df.mean(numeric_only=True))

print("\nStandard Deviation:")
print(df.std(numeric_only=True))
```

Output / expected result

```
DataFrame:
The program prints the first 10 Iris observations with their species names.

Sepal Length Column:
5.1
4.9
4.7
4.6
5.0
5.4
4.6
5.0
4.4
4.9

Mean sepal length for these 10 rows: 4.86 cm.
The complete summary statistics, means and sample standard deviations are printed by the
    program.
```

Result

A DataFrame was created from Iris observations, and column retrieval, summary statistics, mean and sample standard deviation were demonstrated.

Experiment 3: Linear Regression Using CSV Data

Aim

Implement Linear Regression using a training dataset stored in a CSV file and calculate Mean Squared Error (MSE) on test data.

Dataset and notes

Scikit-learn Diabetes dataset (442 rows). BMI is the input feature and disease-progression measure is the target. Training and test partitions are saved to CSV and read back.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the real Diabetes dataset
diabetes = load_diabetes(as_frame=True)
data = diabetes.frame[["bmi", "target"]].rename(
    columns={"bmi": "X", "target": "Y"}
)

# Split the dataset
train, test = train_test_split(
    data, test_size=0.25, random_state=42
)

# Save training and testing data as CSV files
train.to_csv("linear_train.csv", index=False)
test.to_csv("linear_test.csv", index=False)

# Read the CSV files
train_data = pd.read_csv("linear_train.csv")
test_data = pd.read_csv("linear_test.csv")

X_train = train_data[["X"]]
y_train = train_data["Y"]
X_test = test_data[["X"]]
y_test = test_data["Y"]

# Train Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and evaluate
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Predictions:", predictions[:10])
print("Actual Values:", y_test.to_numpy()[:10])
print("Mean Squared Error:", mse)
```

Output / expected result

```
Training Samples: 331
Testing Samples: 111
First 10 predictions: approximately [146.02, 188.07, 148.13, 202.79, 132.36, 128.15, 318.42,
    196.48, 64.03, 167.05]
First 10 actual values: [219, 70, 202, 230, 111, 84, 242, 272, 94, 96]
Test MSE: approximately 3775.6172
```

Result

Linear Regression was trained from CSV files generated from the Diabetes dataset, and test-set MSE was calculated.

Experiment 4: Non-linear (Polynomial) Regression Using CSV Data

Aim

Implement polynomial regression using CSV data and calculate Mean Squared Error.

Dataset and notes

Scikit-learn Diabetes dataset. BMI is the input and disease-progression measure is the target. Degree-2 polynomial features are used.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the real Diabetes dataset
diabetes = load_diabetes(as_frame=True)
data = diabetes.frame[["bmi", "target"]].rename(
    columns={"bmi": "X", "target": "Y"}
)

# Split and save the data
train, test = train_test_split(
    data, test_size=0.25, random_state=42
)
train.to_csv("nonlinear_train.csv", index=False)
test.to_csv("nonlinear_test.csv", index=False)

# Read CSV files
train_data = pd.read_csv("nonlinear_train.csv")
test_data = pd.read_csv("nonlinear_test.csv")

# Create degree-2 polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train = poly.fit_transform(train_data[["X"]])
X_test = poly.transform(test_data[["X"]])
y_train = train_data["Y"]
y_test = test_data["Y"]

# Train polynomial regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and calculate MSE
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)

print("Polynomial Degree: 2")
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Predictions:", predictions[:10])
print("Actual Values:", y_test.to_numpy()[:10])
print("Mean Squared Error:", mse)
```

Output / expected result

```
Polynomial Degree: 2
Training Samples: 331
Testing Samples: 111
First 10 predictions: approximately [144.31, 186.31, 146.33, 201.79, 131.38, 127.47, 337.64,
    195.11, 71.98, 164.89]
First 10 actual values: [219, 70, 202, 230, 111, 84, 242, 272, 94, 96]
Test MSE: approximately 3803.6562
```

Result

Polynomial Regression was implemented using CSV data and evaluated using MSE. On this split, the degree-2 model has a slightly higher MSE than linear regression.

Experiment 5: Logistic Regression and Classification Accuracy

Aim

Implement Logistic Regression using a CSV dataset and compute the classifier's accuracy.

Dataset and notes

Breast Cancer Wisconsin dataset from scikit-learn: 569 samples, 30 numeric features, and two classes (malignant and benign).

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Load the real Breast Cancer dataset
cancer = load_breast_cancer(as_frame=True)
data = cancer.data.copy()
data["Class"] = cancer.target

# Save and reload the CSV dataset
data.to_csv("logistic_regression_data.csv", index=False)
data = pd.read_csv("logistic_regression_data.csv")

X = data.drop(columns=["Class"])
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Scale features and train Logistic Regression
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000, random_state=42)
)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("Predictions:", predictions)
print("Actual Values:", y_test.to_numpy())
print("Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

Output / expected result

```
Training samples: 426
Testing samples: 143
Accuracy: approximately 0.9860
Accuracy Percentage: 98.60%
Confusion Matrix:
[[52  1]
 [ 1 89]]
```

Result

Logistic Regression was trained on the Breast Cancer Wisconsin dataset and achieved approximately 98.60% test accuracy for the specified split.

Experiment 6: Naive Bayesian Classifier

Aim

Implement a Naive Bayes classifier using a CSV training dataset and compute accuracy on test data.

Dataset and notes

Wine recognition dataset from scikit-learn: 178 samples, 13 numeric features, and three classes.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Load the real Wine dataset
wine = load_wine(as_frame=True)
data = wine.data.copy()
data["Class"] = wine.target

# Save and read the CSV file
data.to_csv("naive_bayes_data.csv", index=False)
data = pd.read_csv("naive_bayes_data.csv")

X = data.drop(columns=["Class"])
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Train Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("Predictions:", predictions)
print("Actual Values:", y_test.to_numpy())
print("Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

# Predict three test samples
test_data = X_test.iloc[:3]
test_predictions = model.predict(test_data)
print("\nNew Test Data Predictions:")
for i, prediction in enumerate(test_predictions):
    print("Sample", i + 1, "Predicted Class:", wine.target_names[prediction])
```

Output / expected result

```
Training samples: 133
Testing samples: 45
Accuracy: approximately 0.9778
Accuracy Percentage: 97.78%
Confusion Matrix:
[[15  0  0]
 [ 1 17  0]
 [ 0  0 12]]
```

Result

Gaussian Naive Bayes was implemented using the Wine dataset saved as a CSV file. Test accuracy was approximately 97.78% for the specified split.

Experiment 7: k-Nearest Neighbor Classification of the Iris Dataset

Aim

Implement the k-Nearest Neighbor algorithm to classify Iris and print both correct and wrong predictions.

Dataset and notes

Scikit-learn Iris dataset: 150 samples, four features and three species. The program uses k=3 and a stratified 70/30 split.

Complete Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Scale features and create KNN model
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=3)
)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
correct = 0
wrong = 0

print("KNN Classification Results\n")
for actual, predicted in zip(y_test, predictions):
    status = "Correct" if actual == predicted else "Wrong"
    print(
        "Actual:", iris.target_names[actual],
        "| Predicted:", iris.target_names[predicted],
        "|", status
    )
    if actual == predicted:
        correct += 1
    else:
        wrong += 1

print("\nTotal Test Samples:", len(y_test))
print("Correct Predictions:", correct)
print("Wrong Predictions:", wrong)
print("Accuracy:", accuracy_score(y_test, predictions))
print("Accuracy Percentage:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

Output / expected result

```
The program prints each of the 45 test samples as Correct or Wrong, then prints the calculated
    counts and accuracy. Results are generated by the code at runtime.
```

Result

KNN classification was performed on the Iris dataset with three neighbours. Each prediction is compared with its actual class and classified as correct or wrong.

Experiment 8: Entropy-Based Decision Tree (ID3-Style)

Aim

Demonstrate an entropy-based decision tree and use it to classify a new sample.

Dataset and notes

Scikit-learn Iris dataset. Scikit-learn's entropy criterion demonstrates information-gain splitting, but this is an ID3-style tree, not an exact implementation of original categorical ID3.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Create an entropy-based decision tree
id3_model = DecisionTreeClassifier(
    criterion="entropy", random_state=42
)
id3_model.fit(X_train, y_train)

predictions = id3_model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("Decision Tree Using Entropy")
print("Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")

print("\nDecision Tree Rules:")
print(export_text(
    id3_model,
    feature_names=list(X.columns),
    max_depth=3
))

# Classify a new Iris sample
new_sample = pd.DataFrame([{
    "sepal length (cm)": 5.1,
    "sepal width (cm)": 3.5,
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2
}])
prediction = id3_model.predict(new_sample)[0]

print("New Sample:", new_sample.iloc[0].to_dict())
print("Predicted Class:", iris.target_names[prediction])
```

Output / expected result

```
Test accuracy: approximately 0.8889 (88.89%) for the specified split.
The program prints the learned decision rules and predicts setosa for the new sample.
```

Result

An entropy-based decision tree was trained on Iris, evaluated on test data, and applied to a new sample.

Experiment 9: CART Decision Tree Using Gini Impurity

Aim

Demonstrate the CART decision tree algorithm using a real dataset and classify a new sample.

Dataset and notes

Scikit-learn Wine dataset: 178 samples, 13 numeric features and three wine classes. CART uses binary splits based on Gini impurity.

Complete Python Program

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix

# Load the real Wine dataset
wine = load_wine(as_frame=True)
X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Create the CART classifier using Gini impurity
cart_model = DecisionTreeClassifier(
    criterion="gini", random_state=42
)
cart_model.fit(X_train, y_train)

predictions = cart_model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("CART Decision Tree Results")
print("Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

print("\nDecision Tree Rules:")
print(export_text(
    cart_model,
    feature_names=list(X.columns),
    max_depth=3
))

# Classify a sample with median feature values
new_sample = pd.DataFrame([X.median().to_dict()])
prediction = cart_model.predict(new_sample)[0]

print("\nNew Sample:")
print(new_sample)
print("Predicted Class:", wine.target_names[prediction])
```

Output / expected result

```
For the specified split, test accuracy is approximately 96.30%. The full confusion matrix,
    tree rules, median-valued sample and predicted wine class are printed when the code runs.
```

Result

A CART decision tree using Gini impurity was trained on the Wine dataset and used to classify a new sample.

Experiment 10: Neural Network for AND, OR and XOR Logic Gates

Aim

Implement a neural network and test its working on AND, OR and XOR logic gates.

Dataset and notes

Standard two-input truth tables. These are mathematical examples, not a real-world scikit-learn dataset. Each gate uses the four possible binary input combinations.

Complete Python Program

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# All combinations of two binary inputs
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Truth tables
gates = {
    "AND": np.array([0, 0, 0, 1]),
    "OR":  np.array([0, 1, 1, 1]),
    "XOR": np.array([0, 1, 1, 0])
}

for gate, y in gates.items():
    model = MLPClassifier(
        hidden_layer_sizes=(4,),
        activation="tanh",
        solver="lbfgs",
        max_iter=10000,
        random_state=42,
        tol=1e-8
    )

    # Train the neural network
    model.fit(X, y)

    # Predict outputs
    predictions = model.predict(X)

    print("\n", gate, "Gate")
    print("Input | Expected | Predicted | Status")

    for inputs, expected, predicted in zip(X, y, predictions):
        status = "Correct" if expected == predicted else "Wrong"
        print(inputs, "|", expected, "|", predicted, "|", status)

    accuracy = accuracy_score(y, predictions)
    print("Accuracy:", accuracy)
    print("Accuracy Percentage:", accuracy * 100, "%")
```

Output / expected result

```
The program prints all four inputs, expected and predicted output, status, and accuracy for
    each of AND, OR and XOR. Verify the generated predictions at runtime.
```

Result

A multilayer perceptron was trained separately for AND, OR and XOR and evaluated against each gate's truth table.

Dataset Summary

Dataset	Samples	Features	Experiments	Purpose
Iris	150	4	1, 2, 7, 8	Statistics, DataFrame, KNN, entropy tree
Diabetes	442	10 (BMI used)	3, 4	Linear and polynomial regression
Breast Cancer Wisconsin	569	30	5	Binary Logistic Regression
Wine Recognition	178	13	6, 9	Naive Bayes and CART
Logic-gate truth tables	4 combinations per gate	2 inputs	10	AND, OR, XOR neural network

Required packages

```
%pip install numpy pandas scikit-learn
```